```
when Square, Rectangle
  # ...
when Circle
  # ...
when Triangle
  # ...
else
  # ...
end
```

This example shows another syntactic feature of Ruby's case statement, which is that if you have multiple comparisons for the same result, you can put them in the same when statement and separate them with a comma, as this example does with Square, Rectangle.

There are a couple of other interesting uses of ===. For a Range, === means the target is inside the range. A Set is === if the target is in the set. A Proc checks for === by calling the proc with the target as an argument and using the truth value of whatever the proc returns.

Safe Navigation

It's common to have a chain of method calls on a series of objects. For example, you might want to retrieve a string from a hash and perform other processing on it:

```
data[:name].upcase
```

There's only one problem: You might have no way of knowing if data[:name] returns a value or returns nil. If a value is returned, great! The line of code works as intended. If nil is returned, there's a problem because nil.upcase is not defined and will raise an exception.

As a result, checking for nil is a common pattern:

```
name = data[:name]
if name then name.upcase else nil end
```

This is often written with the && as a shortcut:

```
data[:name] && data[:name].upcase
```

This version, however is a little awkward, and it requires the fetch to be executed twice.

To make this pattern a little nicer, Ruby offers the *safe navigation* operator, &. (ampersand followed by dot). You'll sometimes see this called the lonely operator, because Matz thought that the ampersand dot combination looked like a person sitting alone staring off into space. It works like this:

```
data[:name]&.upcase
```

The way the &. works is that if the receiver of the message on the left side (in this case data[:name]) is nil, then the message is not sent and the nil value is returned without raising an exception. If the receiver is not nil, then the message is processed normally.

This is what we want—the code works as desired for both nil and non-nil values for the data[:name].

The safe navigation operator's powers only last for the one message, if you want to continue with more downstream messages, you need more safe navigation operators.